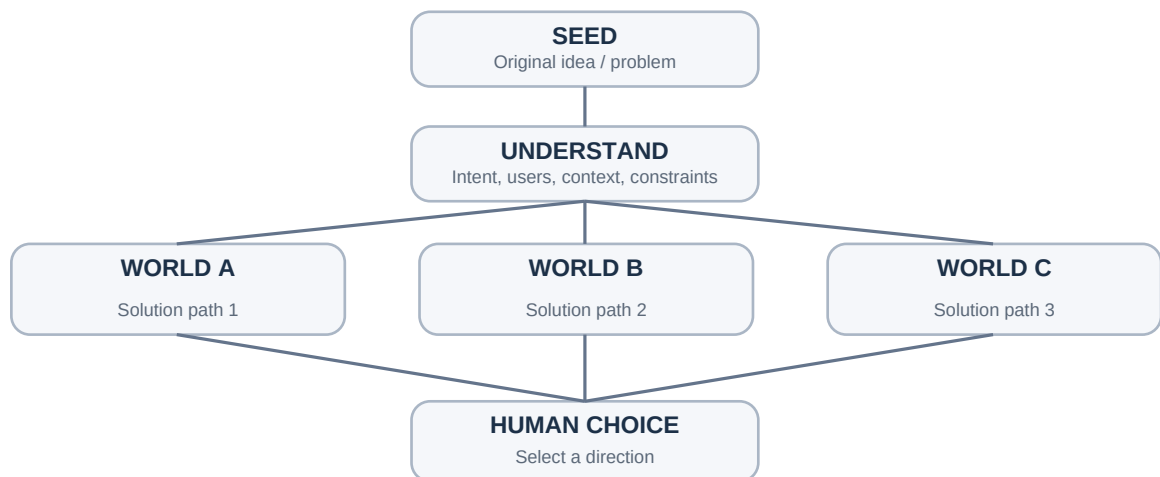


SEED UNFOLD

Complete Project Documentation

From one small idea to multiple possible worlds, a chosen solution, and a traceable final form.



Hackathon Product Blueprint

Concept + problem + solution + USPs + MVP + architecture + AI stack + data model + end-to-end workflow + impact + challenges + implementation + deployment + demo strategy

Version: Hackathon Build Documentation | 01 October 2026

1. Document Purpose

This document is the single source of truth for building and presenting Seed Unfold as a complete, deployable hackathon product. It consolidates the project idea, the Theme 2/Tattva interpretation, the team's documented USPs, the product scope, architecture, AI roles, data structures, implementation sequence, impact, risks, and judge-facing demo.

2. Project at a Glance

Item	Definition
Product	Seed Unfold
Core idea	Turn one incomplete idea or problem (the seed) into multiple structured solution paths, let the human choose one, then progressively unfold it into a buildable solution.
Core metaphor	A compact seed contains larger possibilities that become visible through progressive unfolding.
Theme connection	Tattva 2: Forms hidden in formless. The abstract seed contains latent structure; the system makes those possible forms visible.
Primary users	Students, hackathon teams, early-stage innovators, project builders.
Core differentiators	Progressive Unfolding, Multiple Possible Worlds, Seed DNA / Traceability.
MVP outcome	A stranger can enter a seed, receive 3 branches, choose one, see 5-7 unfolding stages, inspect traceability, and reach a structured solution.

3. Table of Contents

01 Purpose and project definition
02 Problem and opportunity
03 Tattva / Theme 2 connection
04 Solution concept and end-to-end workflow
05 USPs and measurable value
06 Product scope and MVP
07 User experience and screens
08 Technical architecture
09 AI models and provider roles
10 Seed DNA and data model
11 Backend API design
12 Frontend architecture
13 Traceability and explainability
14 Persistence and storage
15 Reliability and fallback strategy
16 Impact and benefits
17 Key challenges and mitigations
18 Security and responsible AI
19 24-hour implementation plan
20 Deployment and production checklist
21 Judge demo flow
22 Future roadmap
23 Definition of done

4. Problem Statement

Great ideas often begin as a sentence, observation, need, or rough problem. The difficulty is not only generating an answer. It is deciding what the idea could become, comparing different directions, and understanding how the final features relate back to the original idea.

Observed gaps

- AI tools can produce plans, requirements, or answers, but users may see only the final artifact rather than the evolution.
- A single seed can support several valid interpretations, but many workflows quickly converge on one dominant answer.
- Weak context can produce generic outputs that are difficult to turn into a concrete project.
- Users often cannot explain why a particular AI-generated feature exists or which original requirement caused it.
- Hackathon teams need a fast bridge from idea to structured, comparable, explainable build direction.

Problem in one sentence

How can we transform a small, incomplete idea into multiple meaningful solution possibilities while keeping the entire transformation understandable and traceable?

Target persona

- Student builder with an idea but limited time.
- Hackathon team that needs several directions before committing.
- Early-stage innovator who wants structure without losing creative choice.
- Mentor, evaluator, or teacher who needs to understand how a solution emerged.

Example seed

"Students struggle to find internships."

Possible paths: Skill Matching, Opportunity Discovery, and Peer Network. The point is not to claim one is universally best. The system makes the alternatives visible so the user can choose a direction deliberately.

5. Tattva / Theme 2 Connection

The project is designed around the theme described in the submitted materials as **Tattva 2: Forms hidden in formless**. The product interprets this as an interaction model: an abstract seed is compact and incomplete, but it contains latent possibilities. Seed Unfold makes those possibilities progressively visible.

Mapping

Theme idea	Seed Unfold interpretation
Formless / compact seed	A sentence, problem, observation, or rough idea.
Hidden possibilities	Intent, users, constraints, requirements, and alternative solution directions.
Unfolding	The system reveals structure in visible stages instead of jumping directly to one final answer.
Multiple forms	Three distinct solution worlds are generated from the same seed.
Human guidance	The user chooses which latent form should continue developing.
Larger final form	The selected branch becomes requirements, features, prototype structure, and optional creative outputs.
Trace to origin	Seed DNA keeps the relationship between the starting idea and later outputs visible.

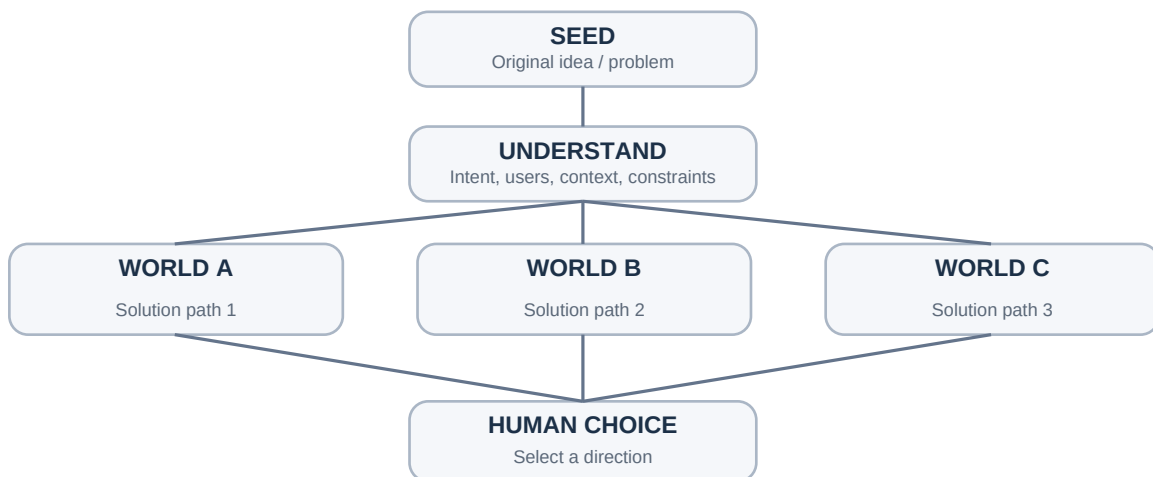
Theme demonstration during judging

The judge should be able to watch one small seed visibly expand into larger forms. The UI should therefore emphasize the transformation, not merely display a chatbot response.

Judge-facing line

"We don't just generate. We show how one small seed unfolds into multiple possible solutions and preserve the connection from the seed to the final form."

6. End-to-End Product Workflow



Core journey

Stage	What happens	Output
1. Seed	User enters an idea/problem.	Raw seed
2. Understand	LLM extracts intent, users, context, constraints, tone, entities.	Seed DNA
3. Branch	LLM creates exactly three meaningfully different solution worlds.	World A / B / C
4. Compare	User reviews the alternatives and their trade-offs.	Human decision
5. Choose	User selects one world.	Selected branch
6. Unfold	System expands the branch through visible stages.	Requirements, features, structure
7. Generate	Optional multimodal outputs are produced from the same Seed DNA.	Story, characters, visual, audio/video
8. Trace	User opens lineage and sees why each major output exists.	Traceability graph
9. Refine	User asks for a refinement or regenerates a stage.	Updated output
10. Branch again	A result can become a new seed for another exploration.	New branch / iteration

Important product principle

Every later stage must receive the relevant Seed DNA and selected branch context. The system should not treat each AI call as an isolated prompt.

7. Unique Selling Propositions

USP	What makes it distinct	Measurable target
1. Progressive Unfolding	Shows the journey from seed to structured solution through 5-7 visible stages.	5-7 visible stages from one seed.
2. Multiple Possible Worlds	One seed produces 3 meaningfully different solution paths before the user commits.	3 branches per seed.
3. Seed DNA / Traceability	Major generated outputs retain a link to the original seed and intermediate requirement nodes.	Target 100% traceability for major outputs.

Why the combination matters

Any single feature can be approximated by existing generative AI. The product identity comes from combining branching, progressive unfolding, and traceability into one continuous experience.

The core positioning

ChatGPT-style systems usually emphasize an answer. Seed Unfold emphasizes the **evolution of an idea**: what it could become, which direction the human chose, and how the final structure relates to the seed.

USP demo moment

Show the same seed at the top of the screen, three worlds beneath it, the selected world expanding through stages, and a final trace graph that links the outputs back to the source.

8. Product Scope: MVP vs Full Hackathon Product

The hackathon MVP is not a half-built application. It is the smallest complete experience that demonstrates the project's core promise end-to-end.

MVP - must work

- Landing page and clear product explanation.
- Seed input with validation.
- Seed understanding and structured Seed DNA.
- Exactly three solution worlds with clear differences.
- Human comparison and branch selection.
- Visible progressive unfolding with 5-7 stages.
- Structured solution output: problem, users, requirements, features, prototype concept.
- Seed DNA context reused across generation steps.
- Traceability graph showing source relationships.
- Persistence of the project and generated results.
- Refine/regenerate action.
- Public deployment with production API configuration.
- Prepared demo seeds and fallback results for external API failure.

Enhanced features - if time remains

- Generated character profiles and visual concepts.
- Image generation for selected world.
- TTS narration.
- Optional music/sound generation.
- Optional short video generation.
- Branch-from-output workflow.
- Export/share project.
- Authentication and user project history.

Not required for the first complete hackathon build

- Complex microservice architecture.
- Custom model training.
- Large-scale user analytics.
- Full social platform.
- Real-time collaborative editing.
- A custom vector database service when simple storage is sufficient.

9. User Experience and Screen Specification

Screen	Required content	Success condition
Landing	Problem, Seed Unfold concept, CTA, short demo visual.	Judge understands the concept in under 20 seconds.
Create Seed	Text input, optional context, example seeds, Unfold button.	User can start without instructions.
Understanding	Intent, users, constraints, theme, extracted entities.	User sees what the AI understood.
Worlds	Three cards with title, approach, target user, differentiator, risks/trade-offs.	User can compare before choosing.
Unfold	Animated stage timeline / cards.	Transformation is visible, not hidden behind a spinner.
Solution	Problem, users, requirements, features, prototype concept.	Output is buildable and structured.
Creative Outputs	Story, characters, visual, audio/video where available.	Outputs visibly share the same world context.
Traceability	Graph of seed -> intent -> requirement -> feature -> output.	User can inspect origin and rationale metadata.
Refine	Natural-language instruction plus regenerate controls.	User can change a result without restarting.
Project View	Saved seed, selected world, outputs, timestamps.	Product survives refresh and can be revisited.

Design direction

Use a cinematic but restrained interface: dark neutral background, strong typography, subtle motion, clear cards, and a central visual metaphor for a seed unfolding. Avoid excessive AI-themed decoration. The interface should make the transformation itself the visual hero.

10. Technical Architecture

Recommended architecture: one React frontend, one FastAPI backend, a simple managed database/storage layer, and provider adapters for AI services. Avoid unnecessary microservices during the 24-hour build.

Logical architecture

React + Vite -> UI / state / animations

FastAPI -> orchestration / validation / persistence / provider adapters

Seed Engine -> Seed DNA creation and branch generation

Generation Engine -> story, structured solution, image/audio/video providers

Storage -> projects, Seed DNA, branches, outputs, trace nodes

Traceability Engine -> parent-child relationships between generated artifacts

Recommended repository

```
seed-unfold/  
  frontend/ # React + Vite + Tailwind  
  backend/ # FastAPI  
  shared/ # JSON schemas / types  
  prompts/ # versioned prompt templates  
  demo/ # prepared seeds and fallback assets  
  README.md  
  .env.example
```

Backend modules

- api/routes_seed.py - seed creation and retrieval.
- api/routes_branches.py - three-world generation and selection.
- api/routes_unfold.py - progressive solution generation.
- api/routes_outputs.py - creative generation and regeneration.
- api/routes_trace.py - lineage graph.
- services/seed_engine.py - Seed DNA construction.
- services/llm_provider.py - LLM adapter.
- services/media_providers.py - image/audio/video adapters.
- services/trace_engine.py - parent-child graph construction.
- db/models.py - persistence models.
- schemas/ - Pydantic request/response contracts.

11. AI Model and Provider Strategy

Use one central reasoning/orchestration model and keep other modalities behind provider interfaces. Exact provider/model availability can change, so model identifiers should live in environment configuration rather than being hard-coded across the application.

Role	Needed?	Purpose	Fallback
Main text LLM	Yes	Seed understanding, branch generation, solution structure, prompt creation.	Prepared JSON outputs for demo mode.
Vision model	Optional	Understand uploaded reference images/documents or visual inputs.	Skip visual input in core flow.
Image generation	Recommended enhancement	Character/world/scene visuals.	Prepared demo images.
TTS	Recommended enhancement	Narration / character voice.	Browser speech or prepared audio.
Music/audio generation	Optional	Atmosphere for selected world.	Prepared audio loop.
Video generation	Optional enhancement	Short cinematic unfold clip.	Prepared video asset.
Embeddings	Optional but useful	Semantic memory / similarity / future retrieval.	Metadata links and relational IDs.

Provider adapter pattern

```
AI_PROVIDER_LLM=...  
IMAGE_PROVIDER=...  
AUDIO_PROVIDER=...  
VIDEO_PROVIDER=...  
EMBEDDING_PROVIDER=...
```

The application should expose one internal interface such as `generate_text()`, `generate_image()`, `generate_audio()`, and `generate_video()`. This lets you replace a provider without rewriting the product.

12. Seed DNA and Data Model

Seed DNA is the internal representation that preserves the identity and constraints of the original seed. It is the context object passed into later generation steps.

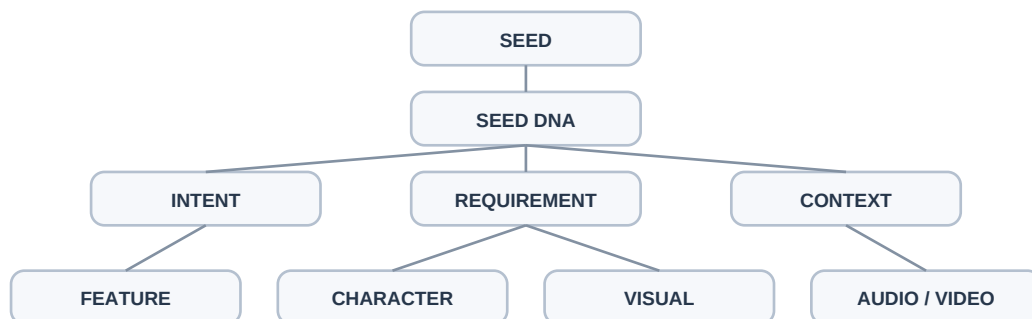
Suggested Seed DNA schema

```
{ "seed_id": "seed_001", "original_text": "...", "intent": "...", "problem": "...", "users": [], "context": [], "entities": [], "constraints": [], "tone": "...", "domain": "...", "visual_style": "...", "keywords": [], "created_at": "..." }
```

Core entities

Entity	Important fields
Project	project_id, title, created_at, updated_at
Seed	seed_id, original_text, seed_dna
Branch	branch_id, seed_id, title, concept, direction, status
Selection	project_id, selected_branch_id, selected_at
Stage	stage_id, branch_id, type, content, status
Asset	asset_id, stage_id, type, uri, provider, metadata
TraceNode	node_id, entity_type, entity_id, parent_id, reason
GenerationJob	job_id, type, status, provider, error, timestamps

Traceability graph



13. Backend API Contract

Keep the API small and predictable. The frontend should never need to know which AI provider generated a result.

Method	Endpoint	Purpose
POST	/api/projects	Create a project.
POST	/api/projects/{id}/seed	Create Seed DNA from raw seed.
POST	/api/projects/{id}/branches	Generate exactly three worlds.
POST	/api/projects/{id}/select	Save human branch choice.
POST	/api/projects/{id}/unfold	Generate structured solution stages.
POST	/api/projects/{id}/outputs	Generate selected creative output.
POST	/api/projects/{id}/refine	Regenerate/refine a selected node.
GET	/api/projects/{id}	Return complete project state.
GET	/api/projects/{id}/trace	Return lineage graph.
GET	/api/health	Deployment health check.

API rule

Every generation endpoint should return structured JSON, not free-form text. Validate LLM outputs with Pydantic schemas and retry once or use a fallback if parsing fails.

14. Persistence, Assets and Reliability

Minimum persistence

- Project and seed record.
- Seed DNA JSON.
- Three branch records.
- Selected branch.
- Generated stage outputs.
- Asset URLs or storage references.
- Trace nodes and parent-child relationships.
- Generation status and errors.

Generation state machine

QUEUED -> GENERATING -> SUCCESS / FAILED -> RETRY / FALLBACK

Reliability strategy

- Never make video generation a prerequisite for the core product journey.
- Prepare 2-3 complete demo projects with cached results.
- Cache expensive generated assets.
- Use timeouts for external providers.
- Show meaningful errors and retry controls.
- If live generation fails, switch to a clearly defined demo/fallback path rather than leaving the UI blocked.
- Keep provider keys on the backend only.

Production behavior

The deployed site must work even when a nonessential multimodal provider is unavailable. Core seed, branching, selection, unfolding, persistence, and traceability should remain usable.

15. Impact and Benefits

User benefits

- Turns vague ideas into structured starting points.
- Encourages exploration before commitment.
- Makes AI output easier to explain and defend.
- Helps teams communicate how a project evolved.
- Reduces the gap between ideation and implementation planning.
- Creates a visual, memorable learning experience for students and hackathon teams.

Educational value

- Teaches problem decomposition.
- Demonstrates requirements-to-feature relationships.
- Shows how constraints influence solution design.
- Encourages human judgment instead of blind acceptance of one AI answer.
- Makes AI orchestration and system design visible.

Longer-term product potential

- Project planning and PRD generation.
- Hackathon ideation and team decision support.
- Innovation workshops and classrooms.
- Startup discovery and early product exploration.
- Design-space exploration for creative and technical projects.

16. Key Challenges and Mitigations

Challenge	Risk	Mitigation
LLM output inconsistency	Invalid or drifting JSON.	Structured schemas, validation, retry, versioned prompts.
Branch similarity	Three worlds become three variations of one answer.	Explicit diversity instructions and post-generation difference checks.
Context drift	Later outputs lose the seed identity.	Inject Seed DNA + selected branch into every generation.
API latency	Judges wait too long.	Parallelize independent jobs, cache assets, show progressive states.
Image/video provider failure	Demo stops at media stage.	Optional media, cached demo assets, provider adapters.
Cost/rate limits	Free tiers are insufficient for repeated generation.	Limit calls, cache outputs, prepared seeds, optional modalities.
Traceability becomes decorative	Graph does not reflect actual generation.	Create trace nodes from the same IDs used by generation records.
Deployment mismatch	Works locally but fails online.	Production URL test before freeze, health endpoint, environment validation.
Scope explosion	Too many features consume hackathon time.	Freeze MVP first; enhancements only after full journey works.

Most important engineering rule

Do not add a feature that can break the complete seed-to-trace demo after the core journey is stable.

17. Security and Responsible AI

- Keep all provider API keys in server-side environment variables.
- Never expose secret keys in the React bundle.
- Validate and sanitize user inputs.
- Limit request size and generation frequency.
- Store only the data needed for the demo/product.
- Treat generated content as AI-assisted rather than guaranteed truth.
- Show source/assumption metadata where useful.
- Do not present generated recommendations as authoritative without human review.

Explainability principle

Seed Unfold should explain *lineage and generation context*, not pretend to reveal hidden private reasoning from the model. The trace graph should show explicit source nodes, inputs, outputs, and reasons stored by the application.

18. Complete 24-Hour Implementation Plan

Time	Phase	Completion target
00:00-00:45	Architecture lock	Repo, environment, schemas, provider adapters, deployment targets chosen.
00:45-02:00	UI shell	Landing, create, unfold, worlds, solution, trace screens visually complete.
02:00-04:00	Seed Engine	Raw seed -> validated Seed DNA.
04:00-06:00	Three Worlds	Exactly three structured branches generated and rendered.
06:00-06:45	Human Choice	Selection persisted and navigation stable.
06:45-09:00	Progressive Unfold	5-7 stages visible and connected.
09:00-13:00	Multimodal generation	Story/characters/visual/audio/video slots integrated with fallbacks.
13:00-14:30	Consistency	Seed DNA and branch context injected into generation.
14:30-16:00	Traceability	Graph and source metadata implemented.
16:00-17:00	Persistence	Projects, stages, assets, trace nodes saved.
17:00-18:00	Refine / branch	Regenerate and branch-from-output working.
18:00-19:00	Hardening	Error, retry, timeout, validation, mobile checks.
19:00-20:00	Deployment	Public frontend/backend/database working.
20:00-21:00	Demo mode	Prepared seeds, cached assets, fallback path.
21:00-22:00	Polish	Motion, typography, visual hierarchy, branding.
22:00-23:00	Judge simulation	Run complete flow as a stranger.
23:00-24:00	Freeze	Critical bugs only, verify URL, APIs, persistence, assets.

Antigravity workflow

Use Antigravity as an implementation accelerator, but feed it phase contracts rather than one giant request.

Phase contract format: Context -> Requirements -> Files to modify -> API contract -> UI requirements -> Acceptance tests -> Stop and verify.

19. Deployment and Production Checklist

Recommended deployment pattern

- Frontend: Vercel or Netlify.
- Backend: Render, Railway, Cloud Run, or equivalent.
- Database: Supabase/Postgres or another managed database.
- Assets: managed object/file storage.
- Secrets: platform environment variables.

Production checklist

- HTTPS enabled.
- CORS restricted to the deployed frontend origin.
- All AI keys present in backend environment.
- Database migrations applied.
- Health endpoint returns success.
- Production API URL configured in frontend.
- No localhost URLs in production build.
- Seed -> branches -> selection -> unfold -> trace tested on production.
- Mobile layout tested.
- External provider failure tested.
- Demo seed tested from a fresh browser session.

20. Judge Demo Flow

The demo should be short, visual, and repeatable. Do not spend the first minutes explaining architecture.

Step	What the judge sees	What you say
1	Landing + one seed	"We start with one small idea."
2	Seed understanding	"The system first makes the hidden structure explicit."
3	Three worlds	"Instead of forcing one answer, it shows three possible directions."
4	Human choice	"The human chooses which possibility should unfold."
5	Progressive unfolding	"Now watch the selected idea expand stage by stage."
6	Final solution	"The result is a structured, buildable solution rather than a single chat response."
7	Traceability	"Every major output remains connected to the original seed."
8	Refine/branch	"The user can refine the result or use it as the starting point for another direction."

Demo rule

If a live external generation call is slow or unavailable, use the prepared demo state. The judge should experience a complete product, not an outage.

Ideal live seed

"Students struggle to find internships."

21. Future Roadmap

Stage	Future capability
V1.1	More branch controls, richer compare mode, project sharing, export.
V1.2	Document/image input and multimodal seed understanding.
V2	Collaborative team ideation and comments.
V2	Versioned solution evolution and branch history.
V2	Stronger semantic retrieval and reusable project knowledge.
V3	Domain-specific agents for product, engineering, design, research, and education.

22. Definition of Done

Seed Unfold is complete for the hackathon when a stranger can open the deployed URL and, without developer intervention, enter a seed, receive three distinct worlds, choose one, watch it unfold through structured stages, inspect the traceability back to the seed, refine or branch the result, and complete the journey reliably.

The final product must therefore prove four things

- The system understands the seed.
- The system explores multiple possible forms.
- The human controls which form develops.
- The system preserves the connection from the original seed to the final solution.

23. Final Product Statement

Seed Unfold is an AI-powered exploration engine that transforms one small idea into multiple possible solution worlds, lets humans choose a direction, progressively develops it into a structured solution, and preserves a visible lineage back to the original seed.

This is the complete product story: **Seed -> Understand -> Branch -> Choose -> Unfold -> Generate -> Trace -> Refine -> Branch again.**

Source basis: team Seed Unfold concept/USP materials and the project architecture discussed in the hackathon planning conversation.